

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Planificación en mini-ciclos

Nuestro enfoque en la formación de nuestros ciclos es avanzar a partir de lo que no depende de nada hacia lo que depende de todo, así tendremos en cuenta que cada uno de los mini- ciclos entregue algo que sea ejecutable y además verificable en BlueJ antes de pasar a realizar el siguiente. También decidimos este orden porque somos conscientes de nuestro nivel de programación y de diseño, entonces decidimos que primero funcione lo básico y solo cuando esta parte sea comprendida por ambos abordaremos la complejidad del problema, como hacerlo robusto.

Mini-ciclo 1 El esqueleto:

Objetivo: Crear las clases del proyecto y lograr que compile. Definiremos la estructura base sin la lógica como tal, la clase principal y sus colaboradoras existen, también que el paquete shapes quede integrado. Es nuestro principal mini-ciclo porque no depende de nada, ninguna operación puede funcionar si no realizamos las clases. Los métodos que abarcaremos será solo el constructor de slotMachine y las clases Wheel y Symbol creadas.

Mini-ciclo 2 Gestionar elementos:

Objetivo: Dar contenido a la maquina como tal, es decir, vamos a agregar y eliminar ruedas, y agregar y eliminar símbolos. La máquina ya puede llenarse, aunque todavía no gira ni se ve. Los métodos que usaremos será addWheel , delWheel, addSymbol, delSymbol, symbols y ok. En este mini- ciclo si tenemos una dependencia y es que depende del anterior ya que las clases deben existir para poder crear y almacenar que agreguemos o queramos eliminar.

Mini – ciclo 3 Dinamismo:

Objetivo: La razón de ser de simular la maquina poner en movimiento la máquina, girando las ruedas, forzando un símbolo en una rueda y consultado el estado. Usaremos los métodos spin, spin(Wheel), placeSymbol, configuration, distinctSymbols e isJackpot. Este ciclo depende del mini-ciclo anterior, ya que gracias a este podremos agregar ruedas y símbolos para poder girarlos y consultarlos.

Mini – ciclo 4 Visual:

Objetivo: Hacer que la maquina se vea, integraremos la representación grafica sobre el modelo ya funcional y extenderemos a Canvas para que reconozca los colores del

estándar CSS como es solicitado en los requisitos. Usaremos makeVisible, makeInvisible, la extensión de Canvas para los colores y el aspecto que debe tener la maquina para verse de manera distinta al estar en un estado ganador.

Mini – ciclo 5 Robustez:

Objetivo: Revisar casos limites y errores que pueden llegar a pasar, en este mini- ciclo rechazaremos entradas que no son validas, este también abarcara la validación de color CSS, el rechazo de un color repetido, las posiciones que estén fuera de rango, cuando ok = false y los mensajes con JOptionPane, que se mostraran únicamente si la maquina se encuentra visible. Este depende de todos los mini- ciclos depende de todos los anteriores.

Mini – ciclo 6 Cierre:

Objetivo: Terminar nuestro simulador de maquina tragamonedas con un cierre, utilizando el método exit, que finalizara el simulador. Este depende de todos los mini- ciclos anteriores.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

En términos de implementación no hemos iniciado ningún mini-ciclo ya que no alcanzamos a escribir código. Lo que completamos fue la etapa previa de diseño, que considerábamos algo indispensable antes de programar, además que tener esto muy bien hecho nos sirvió a imaginar como implementaríamos de manera correcta el código.

La razón de este estado es que esta primera entrega fue muy difícil para nosotros como equipo que apenas comienza, nos tomo la mayor parte del tiempo entender cómo usar Astah, como definir de buena forma el diagrama de clases y comprender que realmente estaban bien los diagramas de secuencia.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Nombre)

Juan Sebastián Garavito Mejía: 10 horas

Laila Zareth Romano Guerrero: 12 horas

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Nuestro mayor logro fue poder construir el diagrama de clases que a nuestro parecer esta de forma coherente con el enunciado, entendimos las decisiones del diseño detrás del diagrama y no solo creamos clases porque si, como pensábamos que era al

principio un diagrama de clases. Lo consideramos el nuestro mayor logro porque el diagrama nos permitió entender que la maquina tragamonedas debe respetar la regla de que todas las ruedas tienen los mismos símbolos en el mismo orden y entender todo esto nos costo mucho, pero nos hizo tener claridad sobre todo el diseño, aunque sigue siendo algo muy abstracto primero diseñar sin tener código y frustra, pero a la final aprendemos.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El manejo de Astah, al ser algo nuevo para nosotros nos costó mucho entender cómo poner los parámetros y los tipos que van en campos separados del nombre, y que Astah genera clases automáticamente al escribir tipos como ArrayList, muchas veces lo borramos y no entendíamos porque se borrada, pero resolvimos aprendiendo a usar las propiedades de la herramienta consultando tutoriales en YouTube, también nos costó mucho separar de que es lo que realmente va en el diagrama y que no, sus relaciones y nombres de estas no las entendíamos para nada, pero pudimos.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Siendo honestos, esta primera entrega fue difícil para nosotros, como equipo logramos comprender el problema, definir los mini-ciclos y construir el diagrama de clases y los diagramas de secuencia, pero no alcanzamos a implementar código. Nos costo entender como usar Astah, como definir de buena forma el diagrama de clases y comprender los diagramas de secuencia y porque se estructuran de esa manera. También nos tomó tiempo entender con claridad que era lo que teníamos que hacer y ponernos de acuerdo como equipo.

Lo que hicimos bien fue realmente no rendirnos ante esas dificultades e insistimos hasta comprender las decisiones de diseño en lugar de dibujar sin entender y terminamos (desde nuestro punto de vista) con un diseño que si refleja el problema. Nos comprometemos a ser mucho mas constantes en el aprendizaje de las c nuevas herramientas, a dedicarles tiempo con anticipación y no sobre la marcha y a enfocarnos en el diseño sin pensar en el código, entendiendo que primero debemos modelar y después si podemos implementar.

7. Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?

La práctica XP más útil para nosotros fue el juego de la planificación, fue la que nos permitió dividir un problema que al inicio ninguno de los dos entendía, que nos parecía

grande y difícil de visualizar de manera real. Entonces en lugar de intentar abordar todo el simulador de la maquina tragamonedas, planificamos incrementos que van desde lo mas simple a lo mas complejo como se puede ver en nuestros mini-ciclos y esto lo consideramos lo más valioso porque le dio forma a nuestro trabajo y nos permitió avanzar de manera un poco mas clara.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Referencias:

- IS-FUNDAMENTOS. (s.f.). *Unidad 1: Control de Versiones (CVS)*. Monitoria de DOPO y MYSD, Escuela Colombiana de Ingeniería Julio Garavito. Recuperado el [fecha de consulta], de <https://fis-eci.github.io/software-foundations-eci/Unidad-CVS/>
- [Lucid [Software](#) [Español](#)]. [***UML Tutorial: Diagrama de Clases desde cero***] . YouTube. Recuperado el [fecha de consulta], de <https://www.youtube.com/watch?v=Z0yLerU0g-Q>

La más útil fue el video de YouTube sobre el diagrama de clases ya que para nosotros fue confuso saber qué relaciones y que tipo de relación debíamos tener en nuestro diagrama de clases y el video fue muy claro para nosotros entender esto.